



King Abdulaziz University

Faculty of Computing & Information Technology Computer Science Department

Final Exam (December 15, 2019)
Course: CPCS-223 (Analysis and Design of Algorithms)
Term: Fall 2019
Points: 100 Time: 120 minutes

Student ID: _____

Student Name: _____

***Section: GA, GE

CLO	SO	Question	Max. Marks	Obt. Marks
05	B	1	10	
05	B	2	20	
11	B	3	20	
04	B	4	20	
08	B	5	20	
15	C	6	10	
		TOTAL	100	

Instructor:

☐ Dr. Maher

☐ Dr Umair

HVCLO – 05, SO - b

(Problem Solving)

Question No. 1

(10 Marks)

For the given set of equations, how the arrays A will be updated during execution of the given algorithm of “Gaussian Elimination”? You are required to reflect all changes to this array after updating a row completely. Also give the final solution of set of equations.

ALGORITHM *BetterForwardElimination*(A[1..n, 1..n], b[1..n])

for $i \leftarrow 1$ **to** n **do** $A[i, n+1] \leftarrow b[i]$

for $i \leftarrow 1$ **to** $n-1$ **do**

$pivotrow \leftarrow i$

for $j \leftarrow i+1$ **to** n **do**

if $|A[j, i]| > |A[pivotrow, i]|$ $pivotrow \leftarrow j$

for $k \leftarrow i$ **to** $n+1$ **do**

$swap(A[i, k], A[pivotrow, k])$

for $j \leftarrow i+1$ **to** n **do**

$temp \leftarrow A[j, i] / A[i, i]$

for $k \leftarrow i$ **to** $n+1$ **do**

$A[j, k] \leftarrow A[j, k] - A[i, k] * temp$

$$x_1 + x_2 + x_3 = 2$$

$$2x_1 + x_2 + x_3 = 3$$

$$x_1 - x_2 + 3x_3 = 8.$$

SOLUTION:

Initial Values:

(01 pts)

A	1	2	3	4
1	1	1	1	2
2	2	1	1	3
3	1	-1	3	8

After Elimination of FIRST Column:

(02 + 02 pts)

Row 2 – 2* row 1

A	1	2	3	4
1	1	1	1	2
2	0	-1	-1	-1
3	1	-1	3	8

Row 3 – row 1

A	1	2	3	4
1	1	1	1	2
2	0	-1	-1	-1
3	0	-2	2	6

After Elimination of Second Column:

(02 pts)

Row 3 – 2* row 2

A	1	2	3	4
1	1	1	1	2
2	0	-1	-1	-1
3	0	0	4	8

Finding Solution by back substitution::

$$4x_3 = 8 \Rightarrow x_3 = 2$$

(01 pts)

$$-x_2 - x_3 = -1 \Rightarrow -x_2 - 2 = -1 \Rightarrow -x_2 = 1 \Rightarrow x_2 = -1$$

(01 pts)

$$x_1 + x_2 + x_3 = 2 \Rightarrow x_1 + (-1) + 2 = 2 \Rightarrow x_1 = 2 - 2 + 1 \Rightarrow x_1 = 1$$

(01 pts)

Question No. 2

20 points

Using the given algorithm (Minimal Requirement Change), generate all permutations for;
S, T, A, R.

(Hint: start generating permutation with smallest character)

If $n = 1$ return 1;

Otherwise

1. generate recursively the list of all permutations of $12...n-1$
2. insert n into each of those permutations by starting with inserting n into $12...n-1$ by moving right to left
3. then switching direction for each new permutation

The permutations must be generated according to the algorithm. Random answers will not be accepted. Show all steps in generating the permutation.

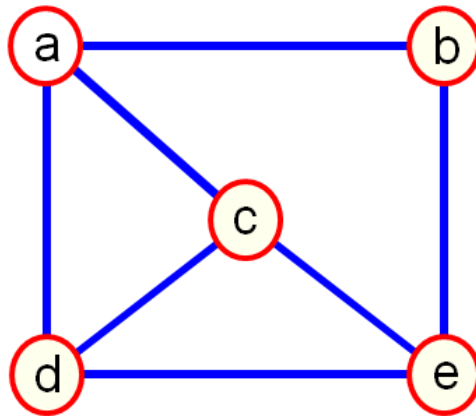
Solution:

Value	Permutations
A (1 pts)	A
R (1.5 for each row)	AR
	RA
S (2 for each row)	ARS, ASR, SAR
	SRA, RSA, RAS
T (2 for each row)	ARST, ARTS, ATRS, TARS
	ASRT, ASRT, ATSR, TASR
	SART, SATR, STAR, TSAR
	SRAT, SRAT, STRA, TSRA
	RSAT, RSTA, RTSA, TRSA
	RAST, RATs, RTAS, TRAS

HVCLO – 11
Question No. 3

SO – B
(20) points

Find the maximum number of paths exist between any two vertices of the given algorithm using the “Transform and Conquer (Problem Reduction)” technique. **The path length is 5 (four)**. You are supposed to show all intermediate results too.



Solution:

Paths of length 01:

(04 pts)

G	a	b	c	d	e
a	0	1	1	1	0
b	1	0	0	0	1
c	1	0	0	1	1
d	1	0	1	0	1
e	0	1	1	1	0

Paths of length 02:

(04 pts)

	a	b	c	d	e
a	3	0	1	1	3
b	0	2	2	2	0
c	1	2	3	2	1
d	1	2	2	3	1
e	3	0	1	1	3

Paths of length 03:

(04 pts)

	a	b	c	d	e
a	2	6	7	7	2
b	6	0	2	2	6
c	7	2	4	5	7
d	7	2	5	4	7
e	2	6	7	7	2

Paths of length 04:

(04 pts)

	a	b	c	d	e
a	20	4	11	11	20
b	4	12	14	14	4
c	11	14	19	18	11
d	11	14	18	19	11
e	20	4	11	11	20

Paths of length 05:

(04 pts)

	a	b	c	d	e
a	26	40	51	51	26
b	40	8	22	22	40
c	51	22	40	41	51
d	51	22	41	40	51
e	26	40	51	51	26

CLO – 4

SO – B

Question No. 4

20 points

Derive a recurrence relation from the given algorithm to calculate the number of addition operations performed during the execution of the algorithm. Also find the solution of the recurrence using any method of your choice studied in class.

ALGORITHM *BinRec*(*n*)

if *n* = 1 **return** 1

else return *BinRec*($\lfloor n/2 \rfloor$) + 1

Solution:

Recurrence Relation:

(08 pts)

$$A(n) = A(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1.$$

$$A(1) = 0.$$

Solution of Recurrence:

(12 pts)

$$A(n) = A(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1.$$

$$A(1) = 0.$$

$$\text{for } n = 2^k$$

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{for } k > 0,$$

$$A(2^0) = 0.$$

Now backward substitutions

$$A(2^k) = A(2^{k-1}) + 1$$

$$= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2$$

$$= [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3$$

...

$$= A(2^{k-i}) + i$$

...

$$= A(2^{k-k}) + k.$$

$$A(2^k) = A(1) + k = k$$

$$\text{substitute } A(2^{k-1}) = A(2^{k-2}) + 1$$

$$\text{substitute } A(2^{k-2}) = A(2^{k-3}) + 1$$

$$n = 2^k \text{ and hence } k = \log_2 n.$$

$$A(n) = \log_2 n \in \Theta(\log n)$$

Compare the following two algorithms in terms of their performance (Complexity) and determine which algorithm is better.

ALGORITHM *BruteForceClosestPair(P)*

$d \leftarrow \infty$

for $i \leftarrow 1$ **to** $n - 1$ **do**

for $j \leftarrow i + 1$ **to** n **do**

$d \leftarrow \min(d, \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2))$

return d

Solution:

(08 pts)

$$\begin{aligned} C(n) &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2 \\ &= 2 \sum_{i=1}^{n-1} (n - i) \\ &= 2[(n - 1) + (n - 2) + \cdots + 1] \\ &= (n - 1)n \\ &= \Theta(n^2) \end{aligned}$$


```

ALGORITHM EfficientClosestPair( $P, Q$ )
if  $n \leq 3$ 
    return the minimal distance found by the brute-force algorithm
else
    copy the first  $\lceil n/2 \rceil$  points of  $P$  to array  $P_l$ 
    copy the same  $\lceil n/2 \rceil$  points from  $Q$  to array  $Q_l$ 
    copy the remaining  $\lfloor n/2 \rfloor$  points of  $P$  to array  $P_r$ 
    copy the same  $\lfloor n/2 \rfloor$  points from  $Q$  to array  $Q_r$ 
     $d_l \leftarrow \text{EfficientClosestPair}(P_l, Q_l)$ 
     $d_r \leftarrow \text{EfficientClosestPair}(P_r, Q_r)$ 
     $d \leftarrow \min\{d_l, d_r\}$ 
     $m \leftarrow P[\lceil n/2 \rceil - 1].x$ 
    copy all the points of  $Q$  for which  $|x - m| < d$  into array  $S[0..num - 1]$ 
     $dminsq \leftarrow d^2$ 
    for  $i \leftarrow 0$  to  $num - 2$  do
         $k \leftarrow i + 1$ 
        while  $k \leq num - 1$  and  $(S[k].y - S[i].y)^2 < dminsq$ 
             $dminsq \leftarrow \min((S[k].x - S[i].x)^2 + (S[k].y - S[i].y)^2, dminsq)$ 
             $k \leftarrow k + 1$ 
    return  $\text{sqrt}(dminsq)$ 

```

Solution:

(08 pts)

$$T(n) = 2T(n/2) + \Theta(n)$$

$$a = 2$$

$$b = 2$$

$$f(n) = n$$

$$n^{\log_b a} = (n^{\log_2 2}) = (n^1) = n$$

Applying 2nd case of Master Theorem
Method

$$\text{If } f(n) = \Theta(n^{\log_b a}), \text{ then } T(n) = \Theta(n^{\log_b a} \lg n)$$

$$T(n) = \Theta(n \lg n)$$

Comparison Results:

“Efficient Closest Pair” is better than “Brute Force Techniques”

(04 pts)

Design an algorithm which receives a BST as a parameter and it returns the difference between the averages of the values of right sub-tree and left sub-tree of the root

EXAMPLE:**Average of Right Subtree**

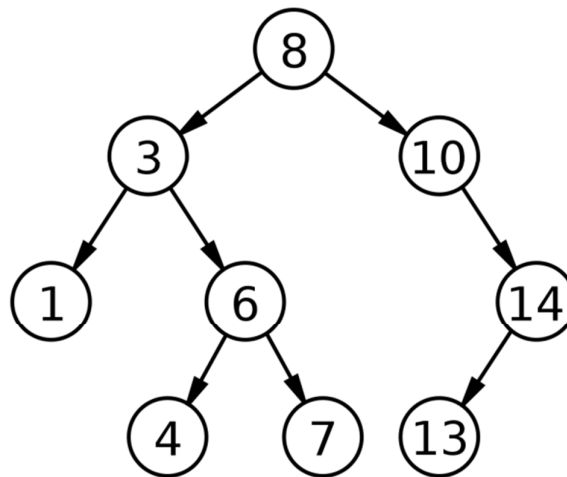
$$=(3+1+6+4+7)/5 = 4.2$$

Average of Left Subtree

$$=(10+14+13)/3 = 12.3$$

Difference

$$= 4.2 - 12.3 = - 8.1$$



If the above tree is passed to the method; it will return “-8.1”

Solution:

```

public int BST(BSTnode p){

    return avgBST (p.right) - avgBST (p.left)          (02 pts)
}
public int avgBST(BSTnode p){

    if (p == null)                                     (01 pt)

        return 0;                                     (01 pt)

    else

        return (p.data + avgBST(p.left)+avgBST(p.right))/          (03 pts)
                (1 + avgBST(p.left)+avgBST(p.right))              (03 pts)

}
  
```

Extra Sheet for rough work